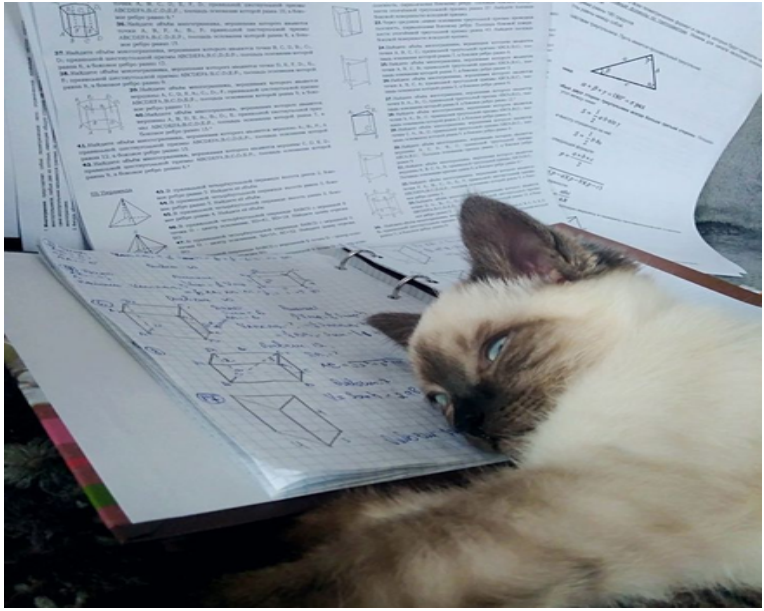


Глава 1. Теория алгоритмов

Ежелев Г. И.

29 августа 2026 г.

Примеры

Понятное дело, что алгоритм, это последовательность действий, которая приводит к вычислению некоторого значения (функции и т.п.). Например: функция от n переменных $(x_1, x_2, ..., x_n)$, которая на выходе возвращает их в порядке возрастания $(x_{i_1} \leq ... \leq x_{i_n})$ - это знакомая вам сортировка массива.

Приведем еще несколько примеров алгоритмических задач:

- Поиск общей подстроки у нескольких входных строк
- Расчет расстояния до объекта на изображении
- Локализация, т.е. расчет положения робота на поле
- Нахождение компонент связности графа
- Нахождение кратчайшего пути в графе (лабиринте).

Основные понятия

Все алгоритмы будут описаны на псевдокоде. Считается, что умение перенести их на конкретный ЯП (язык программирования) - это ремесло.

Любой алгоритм обычно удовлетворяет 3 основным свойствам:

- Корректность - при любых входных данных, результат будет соответствовать требованиям (очев)
- Конечность - нет комбинации входных данных, которые приведут алгоритм к зацикливанию
- Набор команд:
  - Объявление переменных, приравнивание x:=a
  - Условия на $<$, $=$, $>$ и циклы с условиями
  - Операторы goto и return
  - Псевдокод должен в итоге сводиться именно к такому набору операций

Сортировки. Insertion Sort

Основная задача теории алгоритмов - решить задачу оптимальнее очевидного способа.

Очевидный способ отсортировать массив - сортировка пузырьком. Её мы разбирать не будем, это зло.

⊡ Вход: массив $(x_1, ..., x_n)$

⊡ Задача: Отсортировать массив по возрастанию. ⊡ Описание алгоритма:

- Алгоритм напоминает сортировку колоды игральных карт.
- Массив делится на две части: отсортированная и еще нет. В начале отсортированная часть - это один элемент x_1
- Переход i-1 -> i: первые i-1 элементов отсортированы. Хотим вставить i-ый элемент в отсортированную часть

Алгоритм 1 Insertion Sort

Вход: Массив A:=$x_1, ..., x_n$

```
 1: function QuickSort(A)
 2:     for i = 2 ... n do                ▷ Цикл по длине отсортированной части
 3:         x := A[i]          ▷ Запомнили первый неотсортированный элемент
 4:         while j ≥ 1 & x < A[j] do                ▷ Перебираем колоду карт
 5:             A[j+1] = A[j]                        ▷ Освобождаем место под X
 6:             j := j-1
 7:         end while
 8:         A[j+1] = x                ▷ Вставляем X в освободившееся место
 9:     end for
10: end function
```

Сложность алгоритма.

Оптимальность алгоритма можно оценивать через его асимптотическую сложность. Для этого введем несколько определений:

- Сложность алгоритма - количество его итераций (повторений действий) или количество действий, для расчета результата.
- Сложность алгоритма сложно расчитать явно, а вот прикинуть можно через O-нотацию (Big-O)
- Пусть f и g - функции. g(n) := O(f(n)), если для некоторой константы C > 0 $g(n) \leq C * f(n)$
- То есть функция g всегда ограничена сверху функцией f, умноженной на константу.
- Т.к. время исполнения одной итерации цикла расчитать сложно (действительно, не очевидно сколько времени займет операция j = j-1 и т.п.), то мы можем принять это время за некоторую константу C. А вот количество итераций цикла посчитать обычно проще.
- Именно поэтому и используется Big-O она позволяет отбросить эту константу и смотреть лишь на "порядок"сложности.

Сложность Insertion Sort.

Корректность алгоритма очевидна, давайте оценим сложность. В данном случае время алгоритма очень сильно зависит от "степени упорядоченности"исходного массива

Лемма 1

⊡ Лучшее время: $O(n)$

⊡Худшее время: $O(n^2)$

▷

•Пусть массив уже отсортирован. Тогда внешний цикл пробежит каждый элемент, а внутренний цикл на первой же итерации получит противоречие с условием $x \leq A[j]$ и не будет сдвигать массив.

•Таким образом получим n итераций => сложность $\boxed{O(n)}$

Сложность Insertion Sort.

Лемма 1

⊡ Лучшее время: $O(n)$

⊡Худшее время: $O(n^2)$

•Рассмотрим самую грубую худшую оценку: пусть внутренний цикл каждый раз будет пробегать "почти всю"отсортированную часть.

•Внешний цикл пробежит весь массив в любом случае. Таким образом на каждую из n итераций внешнего цикла мы получили "почти n"итераций внутреннего.

•Тогда всего $\approx n^2$ итераций => сложность $\boxed{O(n^2)}$

•Наихудший случай соответствует массиву, отсортированному наоборот, т.е. по убыванию.

◁

Чтобы сравнить скорость алгоритмов, достаточно сравнить их Big-O нотации

InsertionSort - очень простой, но не очень быстрый алгоритм. Если скорость роляет, лучше использовать подход ниже.

Продвинутая сортировка. MergeSort

Merge Sort - Сортировка слиянием использует подход - "Разделяй и властвуй". Разделим задачу на подзадачи, которые по-отдельности решать быстрее, чем вместе, а лучше подзадачи разделим на под-подзадачи, и так до тех пор, пока решение не станет тривиальным.
⊡Описание алгоритма:

- Разделим массив пополам и рекурсивно вызовем алгоритм для каждой половинки, когда рекурсия закончится - получим отсортированные половинки
- Далее их надо слить воедино.
- Это можно сделать за линейное время: поставим указатели на начало каждой половинки. Пусть это указатели p и q. Также добавим указатель в массиве-результате
- Если A[p] <= A[q], то добавим A[p] в A[r]: A[p] := A[r], иначе A[q]
- Когда одна половинка закончится, вся остальная будет добавлена после A[r], т.е. в конец массива.

Алгоритм 2 Merge Sort

Вход: Массив A:=$x_1, ..., x_n$

Выход: Отсортированный массив R

1: **function** MERGE_SORT(A)
2: MERGE_SORT($(x_1, ..., x_{[\frac{n}{2}]})$)
3: MERGE_SORT($(x_{[\frac{n}{2}]+1}, ..., x_n)$)
4: MERGE($(x_1, ..., x_{[\frac{n}{2}]}), (x_{[\frac{n}{2}]+1}, ..., x_n)$)
5: **end function**
6: ————
7: **function** MERGE(A_0, A_1)
8: Пусть p_0, q_0 — указатели на начала половинок
9: Пусть p_1, q_1 — указатели на концы половинок
10: Пусть $p := p_0, q := q_0, r = 0$ — текущий элемент

N.B. Естественно рекурсию надо ограничить, самый очевидный вариант - разбитые половинки состоят из одного элемента, тогда их можно вернуть сразу, без процедуры merge.
Отметим, что алгоритм сразу меняет массив R, поэтому нету return.

```
11:     while p < p_1 & q < q_1 do
12:         if A_0[p] ≤ A_1[q] then
13:             R[r] := A_0[p]
14:             p++
15:             r++
16:         else
17:             R[r] := A_1[q]
18:             q++
19:             r++
20:         end if
21:     end while
22: end function
```

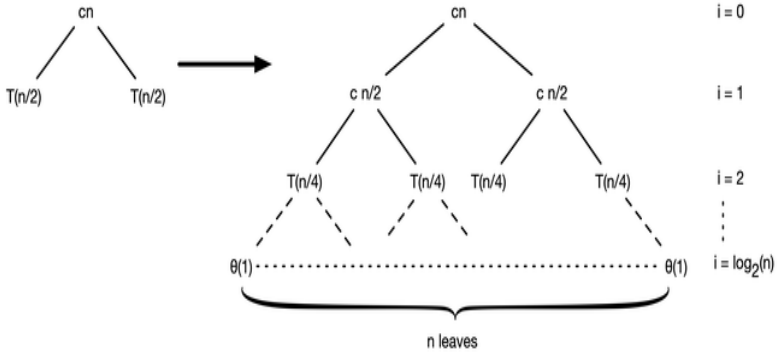
Лемма 2

Сложность Merge Sort. $O(n * \log_2(n))$

▷

•Алгоритм действует по рекурсии. Посчитаем количество действий на каждом уровне рекурсии.

•Вообще говоря, функция merge_sort просто делит массив, никаких действий она не выполняет. Значит она делает почти нулевой вклад в итоговую сложность алгоритма.
•На первом (исходном) уровне операция Merge пробегается циклом по каждому элементу каждой половинки ровно один раз. Т.е. ровно n итераций.
•На втором уровне длина каждой половинки $\frac{n}{4}$, таких половинок тоже 4
•На уровне рекурсии i получим 2^i половинок, каждая длины $\frac{n}{2_i}$.
•Всего $2^i * \frac{n}{2_i} = n$ операций на каждом уровне.
•А сколько всего уровней?
•Массив при рекурсивных вызовах разбивается на неполное бинарное дерево, его высота $\log_2 n$.
•Действительно, на самом нижнем уровне каждая вершина дерева - ровно один элемент массива, а на i-м уровне максимум 2^i вершин. Далее высота дерева мгновенно следует из того, что на нижнем уровне должен поместиться весь массив(см. картинку ниже)
◁



•На самом деле можно доказать, что алгоритм сортировки не может гарантированно отсортировать массив за менее чем $n * \log_2 n$ сравнений элементов, ему просто не хватит информации. Более быстрые алгоритмы используют еще что-то кроме сравнений <,>.
•Если хотите еще поизучать тему, то почитайте про QuickSort.
Задание: Реализуйте один из двух алгоритмов на C++ в любой ide.

Optional: Минимальное число сравнений в сортировках

Теорема 1

Любому алгоритму сортировки, использующему только операции $<$, $>$, $\leq$ и $\geq$, необходимо не менее $O(n * \log_2(n))$ сравнений.

$\triangleright$

•Обозначим последовательность элементов исходного массива $(x_1, ..., x_n)$ номерами индексов самих элементов: $(1, ..., n)$, так удобнее.

•Номера присваиваются элементам в исходном массиве, далее мы можем менять эти индексы местами

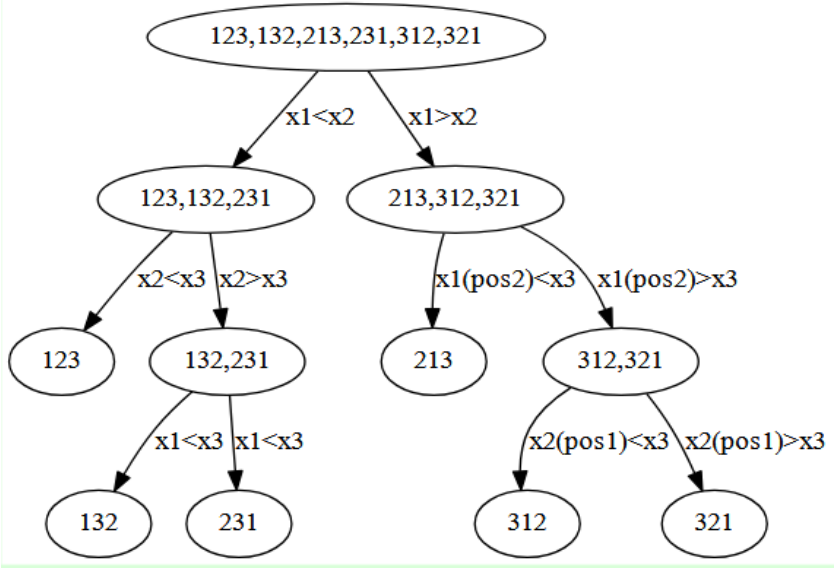
•В самом начале мы не знаем, какой будет последовательность номером индексов в отсортированном массиве. Причем любая последовательнсть чисел может подойти.

•Посчитаем число таких возможных результатов:

- На первом месте может быть любое число от 1 до n
- На втором все, кроме одного, т.е. n-1 вариантов

  ...
- На последнем месте вариант будет ровно 1.
- Итого $n * (n-1) * (n-2) * ... * 1 = n!$

•То есть в самом начале у нас n! вариантов.
•При каждом сравнении мы делим все возможные варианты расположений элементов в массиве на 2:
Пусть $x_i \leq x_j$ тогда i должно быть левее j. Ясно что последовательностей где i левее j и последовательностей, где j левее i поровну, отсюда и деление на 2.
•Представим каждое такое деление как бинарное дерево из предыдущего алгоритма.
•Мы знаем, что его высота $log_2(n!) = log_2(n * (n-1) * ... * 1)$
•Мы можем грубо оценить, что
$n! \leq n^n => \log_2 n! \geq \log_2(n^n) - O(n) = n * \log_2(n) - O(n)$
•Тогда сложность: $O(n * \log_2(n) - O(n)) = O(n * (\log_2(n) - C)) = O(n * (\log_2(n)),\ C > 0$ - некоторая константа
◁
•На самом деле, предпоследний переход стоит доказывать из оценки $(\frac{n}{e})^n \leq n! \leq n^n$, но это уже не так просто.



•Здесь числа - это сами элементы массива, а не их индексы. А вот x_i - i-ое число.

- Таким образом мы доказали, что MergeSort один из самых эффективных алгоритмов сортировки, использующих только сравнения, он максимально приблизился к нижней возможной границе.
- Если использовать что-то кроме сравнений, то можно добиться сложности примерно O(n), например: Radix Sort - сортировка по разрядам числа.

Максимальная подстрока

- Строка - это конечная последовательность некоторого алфавита. Обозначать будем так: $A := a_1 a_2 ... a_n$
- Чтобы найти общую подстроку у двух строк A и B нужно установить два указателя в начало строк, далее смещать их на 1 вправо и проверять $a_i \stackrel{?}{=} b_i$.
- Если $a_i = b_i$ добавляем этот знак к промежуточной строке и увеличиваем счётчик на 1
- Если $a_i \neq b_i$ Сравниваем текущую промежуточную строку с максимальной найденной и, возможно, записываем промежуточную вместо максимальной и сбрасываем.
- Такой метод (с двумя указателями, сравнивающими объекты) называется метод двух указателей.
- Время работы: $\boxed{O(n)}$

Максимальная подпоследовательность в строках

•Более сложная задача: найти максимальную подпоследовательность у строк $A := a_1 a_2 ... a_n$ и $B := b_1 b_2 ... b_n$.
Т.е. найти такие индексы $1 \leq i_1 < i_2 < ... < i_k \leq n$ и $1 \leq j_1 \leq j_2 < ... < j_k \leq n$, что $a_{i_1} ... a_{i_k} = b_{j_1} ... b_{j_k}$

•Для этого нам понадобится метод динамического программирования: будем решать задачу снизу вверх. Будем пробегаться по строке двумя указателями. Когда найдем общий элемент, присоеденим к наибольшей подпоследовательности, которую мы нашли в оставшейся слева от указателей части строки.

Народный алгоритм нахождения общей подпоследовательности

⊡ Описание алгоритма:

- Будем заполнять квадратную таблицу (или, если угодно - матрицу) T. Число на пересечении i-ой строки и j-го столбца обозначим $T_{i,j}$
- Длина строки A - m; B - n. Алгоритм будет записывать в элемент $T_{i,j}$ максимальную подстроку для строк $a_1a_2...a_i$ и $b_1b_2...b_j$, размер таблицы $(m+1)*(n+1)$.
- Сначала заполним верхнюю строку $T_{0,j}, j \in 1, ..., n$ нулями (действительно, длина подстроки A равна 0).
- Сделаем то же самое с крайним левым столбцом $T_{i,0}$
- Теперь заведем два цикла for, первый будем пробегаться по строке A. Внутри него будет второй цикл, который будет пробегаться по строке B.
- Тогда суммарно все итерации внутреннего цикла пробегут все возможные пары индексов (i, j) у строк A и B соответственно.
- A значит внутренний цикл пробежит и всю таблицу T.

- Теперь достаточно описать процесс заполнения таблицы на каждой итерации внутреннего цикла:
- Если символы $a_i = b_j$ то мы хотим присоеденить их к максимальной подпоследовательности строк $a_1...a_{i-1}$ и $b_1...b_{j-1}$ и получить максимальную подстроку среди строк длины i и j соответственно.
- В таком случае очевидно $T_{i,j} := T_{i-1,j-1} + 1$
- Остался случай, когда $a_i \neq b_j$.
  •Тогда максимальная подпоследовательность не увеличивается. Значит в максимальной подпоследовательности либо не использован a_i, либо b_j.
- Тогда $T_{i,j} := max(T_{i-1,j}; T_{i,j-1})$.
- После запуска алгоритма длина максимальной подпоследовательности будет лежать в элементе $T_{m,n}$
- Чтобы восстановить саму подпоследовательность понадобится несложный обратный спуск по таблице (см. алгоритм)

Алгоритм 3 Народный алгоритм нахождения подпоследовательности

Вход: строки A, B; $|A| = m, |B| = n$

```
1:  for j = 0,...,n do
2:      T_{0,j} := 0
3:  end for
4:  for i = 1,...,m do T_{i,0} := 0
5:      for j = 1,...,n do
6:          if a_i == b_j then
7:              T_{i,j} = T_{i-1,j-1} + 1
8:          else
9:              T_{i,j} = max(T_{i-1,j}, T_{i,j-1})
10:         end if
11:     end for
12: end for
```

```
13: i, j := m, n
14: R := <пустая строка>
15: while i > 0&j > 0 do —Цикл восстановления подстроки
16:     if a_i == b_j then
17:         R+ = a_i
18:         i, j := i-1, j-1
19:     else
20:         if T_{i,j-1} >= T_{i-1,j} then
21:             j − −
22:         else
23:             i − −
24:         end if
25:     end if
26: end while
```

Лемма 3
Сложность алгоритма 3 - $O(mn)$

$\triangleright$

- Сначала разберем цикл заполнения таблицы.
- Каждый элемент таблицы T заполняется ровно один раз. Значит сложность этого цикла $O(mn)$, или $O(n^2)$, если $m = n$.
- Верхний цикл for действует за $O(n)$. Можете сами убедиться что при $n > 1$ имеем $n^2 > n \implies O(n^2 + n) = O(n^2)$
- Нижний цикл, восстанавливающий подстроку действует за $O(n + m)$, что аналогично предыдущему пункту не даёт вклад в итоговую оценку.

$\triangleleft$

Следствие 1

$O(n^m + n^k) = O(n^{max(m,k)})$
$O(n^m + n^m) = O(2 * n^m) = O(n^m)$

•Если вам понравилось разбираться в алгоритмах не с прикладной, а с концептуальной точки зрения, советую почитать:
конспект лекций по алгоритмам